Lecture 4: Parallel Programming Models

Shared memory programming with OpenMP uses threads that all access the same memory. A parallel region starts with the pragma omp parallel directive, and loops are split across threads with omp parallel for. Race conditions happen when two threads write the same variable without synchronization; use critical sections, atomics, or reductions to avoid them.

The reduction clause, for example reduction(+:sum), gives each thread a private copy of the variable and combines the copies at the end of the loop.

Distributed memory with MPI

MPI (Message Passing Interface) runs separate processes, each with its own memory. Processes communicate by sending messages. MPI_Send and MPI_Recv are blocking point-to-point operations. MPI_Bcast sends data from one root process to all others, and MPI_Reduce combines values from all processes onto the root.

Every MPI program starts with MPI_Init and ends with MPI_Finalize. MPI_Comm_rank gives the id of the current process and MPI_Comm_size gives the number of processes.

Performance: Amdahl's law

Amdahl's law says the speedup of a program is limited by its serial fraction. If 10 percent of the program is serial, the maximum speedup is 10 times, no matter how many processors are used. Gustafson's law instead assumes the problem size grows with the number of processors.

Exam information: the written exam is on December 18 in exam hall B. Bring a calculator and a pen.